

ReviewPulse

Hackathon Role-Wise Study Guide - 6 Team Members

Common pitch: ReviewPulse converts raw customer reviews into explainable product insights while protecting customer privacy.

Person 1 - Product Story and Pitch Lead

Study files: README.md and overall project flow

What to study

- Problem: customer reviews are large, unstructured, and difficult to analyse manually.
- Solution: ReviewPulse converts reviews into sentiment, themes, evidence, privacy-safe text, and trends.
- Know the 30-second pitch and the 2-minute presentation sequence.
- Explain the business value: faster product decisions and early complaint detection.

Demo responsibility

Open the dashboard, explain the problem, show the bundled 10K review dataset, and connect every chart to a product decision.

Likely judge questions

Why is this useful? Who uses it? What decisions can a product team make from the output?

Person 2 - Streamlit Dashboard and UX Lead

Study files: app.py

What to study

- CSV upload and automatic column detection for review, review_text, rating, date, and app_name.
- Dashboard metrics: reviews analysed, positive signals, negative signals, PII protected, and validation accuracy.
- Sentiment landscape, theme chart, evidence expanders, mood-over-time chart, and audit table.
- How the dashboard calls analyze_reviews(records) and displays its result.

Demo responsibility

Own the live UI walkthrough: upload a CSV, point out the charts, expand evidence, and show the audit trail.

Likely judge questions

What input format is supported? Can users analyse their own data? What happens if a column is missing?

Person 3 - AI, Sentiment, and Theme Analytics Lead

Study files: sentiment.py, themes.py, and review_pipeline.py

What to study

- Sentiment uses an offline, explainable word lexicon. Positive and negative signal words produce a bounded score.
- Themes use keyword groups such as reliability, performance, usability, support, price, privacy, and features.
- Every classification retains the matching signal words, making the output traceable.
- Know the limitation: this is a hackathon model; production needs human-labelled data and a tested multilingual model.

Demo responsibility

Expand an evidence item and explain exactly which words triggered the sentiment and theme classification.

Likely judge questions

Is this machine learning? How is explainability achieved? How would you improve accuracy for production?

Person 4 - Privacy and Responsible AI Lead

Study files: pii_logic.py and the privacy parts of app.py

What to study

- PII is redacted before the text is analysed or displayed.
- Supported patterns: email addresses, phone numbers, card-like numbers, and URLs.
- Know the replacements: [EMAIL REDACTED], [PHONE REDACTED], [CARD REDACTED], and [URL REDACTED].
- The dashboard audit trail displays protected text, not the original sensitive values.

Demo responsibility

Use a sample review containing an email or phone number and show that the displayed result is redacted.

Likely judge questions

When does redaction happen? What PII is covered? What would you add for production?

Person 5 - Backend and API Integration Lead

Study files: main.py and review_pipeline.py

What to study

- FastAPI exposes GET /health, GET /api/clean-review, and POST /api/analyze.
- The API and Streamlit dashboard share the same analysis pipeline, so logic is not duplicated.
- Know the example JSON review object and the empty-payload validation response.
- Run command: uvicorn main:app --reload.

Demo responsibility

Show how another frontend or service could send reviews to POST /api/analyze and receive structured results.

Likely judge questions

How can this integrate with a mobile app or existing frontend? Why expose an API? How is invalid input handled?

Person 6 - Testing, Validation, and Quality Lead

Study files: test_pipeline.py and quality logic in review_pipeline.py

What to study

- Invalid rows are skipped and counted instead of breaking the full batch.
- Validation uses sentiment_label when available, otherwise a rating-derived proxy: 4-5 positive, 3 neutral, 1-2 negative.
- Monthly sentiment trend is generated from dated reviews.
- Jensen-Shannon divergence compares the first and second time periods and reports Stable or Watch.
- Run command: .venv/bin/pytest.

Demo responsibility

Show the quality and model notes section, explain validation accuracy, and mention the difference between a proxy label and ground truth.

Likely judge questions

How do you know the model is correct? What happens with bad rows? How do you detect changing customer mood?

Final team checklist

- Everyone should know the overall pitch, privacy flow, model limitation, and how to run the project.
- Dashboard: `streamlit run app.py`
- API: `uvicorn main:app --reload`
- Tests: `.venv/bin/pytest`
- Do not call rating-derived labels ground truth; clearly describe them as a proxy.